

ALGORITHME lemmatiseur_malagasy(mot)

ENTREE: mot (chaîne)

SORTIE: lemme, prefixe, suffixe, racine

// Vérifier les exceptions d'abord

SI mot ∈ exceptions_dictionnaire ALORS

 lemme = exceptions_dictionnaire[mot]

 RETOURNER (lemme, null, null, lemme)

FIN SI

// Étape 1: Détection des suffixes

suffixe = detecter_suffixe(mot)

SI suffixe ≠ null ALORS

 racine_candidate = mot[0:longueur(mot)-longueur(suffixe)]

 SI est_racine_valide(racine_candidate) ALORS

// Garder pour analyse des préfixes

 mot_intermediaire = racine_candidate

 FIN SI

SINON

 mot_intermediaire = mot

FIN SI

// Étape 2: Détection des préfixes

prefixe = detecter_prefixe(mot_intermediaire)

SI prefixe ≠ null ALORS

 racine_candidate = mot_intermediaire[longueur(prefixe):]

 SI est_racine_valide(racine_candidate) ALORS

 racine = racine_candidate

SINON

 racine = mot_intermediaire

FIN SI

SINON

 racine = mot_intermediaire

FIN SI

// Étape 3: Déterminer le lemme

SI racine ≠ mot ALORS

 lemme = racine

SINON

 lemme = mot

FIN SI

RETOURNER (lemme, prefixe, suffixe, racine)

FIN ALGORITHME

ALGORITHME detecter_suffixe(mot)

ENTREE: mot

SORTIE: suffixe ou null

suffixes = [

 'ana', 'an', 'ina', 'in', 'na', 'n',

 'tra', 'ka', 'ty', 'ika', 'iko'

]

POUR CHAQUE suffixe DANS suffixes FAIRE

 SI mot.endsWith(suffixe) ET longueur(mot) > longueur(suffixe) + 2 ALORS

 RETOURNER suffixe

FIN SI

FIN POUR

RETOURNER null

FIN ALGORITHME

ALGORITHME detecter_prefixe(mot)

ENTREE: mot

SORTIE: prefixe ou null

prefixes = [

'mi', 'ma', 'man', 'mam', 'maha',

'mpan', 'mpam', 'f', 'fan', 'fam',

'fi', 'ha', 'ka', 'ta', 'm'

]

POUR CHAQUE prefixe DANS prefixes FAIRE

SI mot.startsWith(prefixe) ET longueur(mot) > longueur(prefixe) + 2 ALORS

RETOURNER prefixe

FIN SI

FIN POUR

RETOURNER null

FIN ALGORITHME

ALGORITHME est_racine_valide(racine)

ENTREE: racine candidate

SORTIE: booléen

// Condition 1: Longueur minimale

SI longueur(racine) < 2 ALORS

RETOURNER faux

FIN SI

// Condition 2: Contient au moins une voyelle

SI NON contient_voyelle(racine) ALORS

RETOURNER faux

FIN SI

// Condition 3: Ne commence pas par des combinaisons interdites

interdits_debut = ['nb', 'nk', 'mk', 'dt', 'bp', 'sz']

POUR CHAQUE interdit DANS interdits_debut FAIRE

SI racine.startsWith(interdit) ALORS

RETOURNER faux

FIN SI

FIN POUR

// Condition 4: Respecte les structures syllabiques de base

SI NON valider_structure_syllabique(racine) ALORS

RETOURNER faux

FIN SI

RETOURNER vrai

FIN ALGORITHME